Data Visualization Lab (TE7663)

Practical 7

Integrating R with Power BI: Airline Passenger Analysis and 1961 Forecast

Detail	Value
Name	Priyansh Pandey
PRN	24070126143
Class and batch	TY AI-ML B (B-3)
Institute	Symbiosis Institute of Technology
Course	Data Visualization Lab (TE7663)
Tools used	R 4.6, RStudio, Power BI Desktop
Dataset	AirPassengers (built into R)
Blog post	https://github.com/ppriyansh-05/BLOG--DATA-VISUALIZATION-

1. Aim

The aim of this practical is to connect R with Power BI Desktop and use it for an advanced analytics use case. R is used to load and clean the data, work out summary statistics, build a seasonal ARIMA forecast and draw four R visuals. Power BI is then used to bring everything together in one interactive dashboard with KPI cards, slicers, filters and drill-down.

2. About the dataset

AirPassengers is a classic time series that ships with R. It holds the total number of international airline passengers per month from January 1949 to December 1960. That gives 144 monthly values and every value is in thousands of passengers. The series has two clear features which make it a good fit for this practical. Traffic keeps growing year after year and there is a strong summer peak every year that grows along with the trend.

The business question for this dashboard is simple. How fast is air traffic growing, which months put the most pressure on capacity and how many passengers should the airline plan for in 1961?

3. Task 1: Installing R and setting it up in Power BI

R 4.6 and RStudio were installed first. The three packages used in this practical were then installed from the RStudio Console. forecast is used for the ARIMA model, ggplot2 for the charts and dplyr for data handling.

```
install.packages(c("forecast", "ggplot2", "dplyr"))
```

```
> install.packages(c("forecast", "ggplot2", "dplyr"))
WARNING: Rtools is required to build R packages but is not currently installed. Please download and install th
e appropriate version of Rtools before proceeding:

https://cran.rstudio.com/bin/windows/Rtools/

Installing packages into 'C:/Users/RRNar/AppData/Local/R/win-library/4.6'
(as 'lib' is unspecified)
also installing the dependencies 'cpp11', 'farver', 'labeling', 'RColorBrewer', 'viridisLite', 'utf8', 'pkgcon
fig', 'colorspace', 'fracdiff', 'generics', 'lmtest', 'magrittr', 'Rcpp', 'timeDate', 'urca', 'withr', 'zoo',
'RcppArmadillo', 'cli', 'gtable', 'isoband', 'lifecycle', 'rlang', 'S7', 'scales', 'vctrs', 'glue', 'pillar',
'R6', 'tibble', 'tidyselect'
```

Figure 1: Installing the forecast, ggplot2 and dplyr packages in the RStudio Console

The Rtools warning in the output can be ignored. Rtools is only needed when a package has to be built from source and these packages install as ready made binaries on Windows.

In Power BI Desktop the R home folder was checked under File > Options and settings > Options > R scripting. Power BI picked up the R 4.6 install on its own. The first time an R visual was added Power BI asked to enable script visuals and this was allowed.

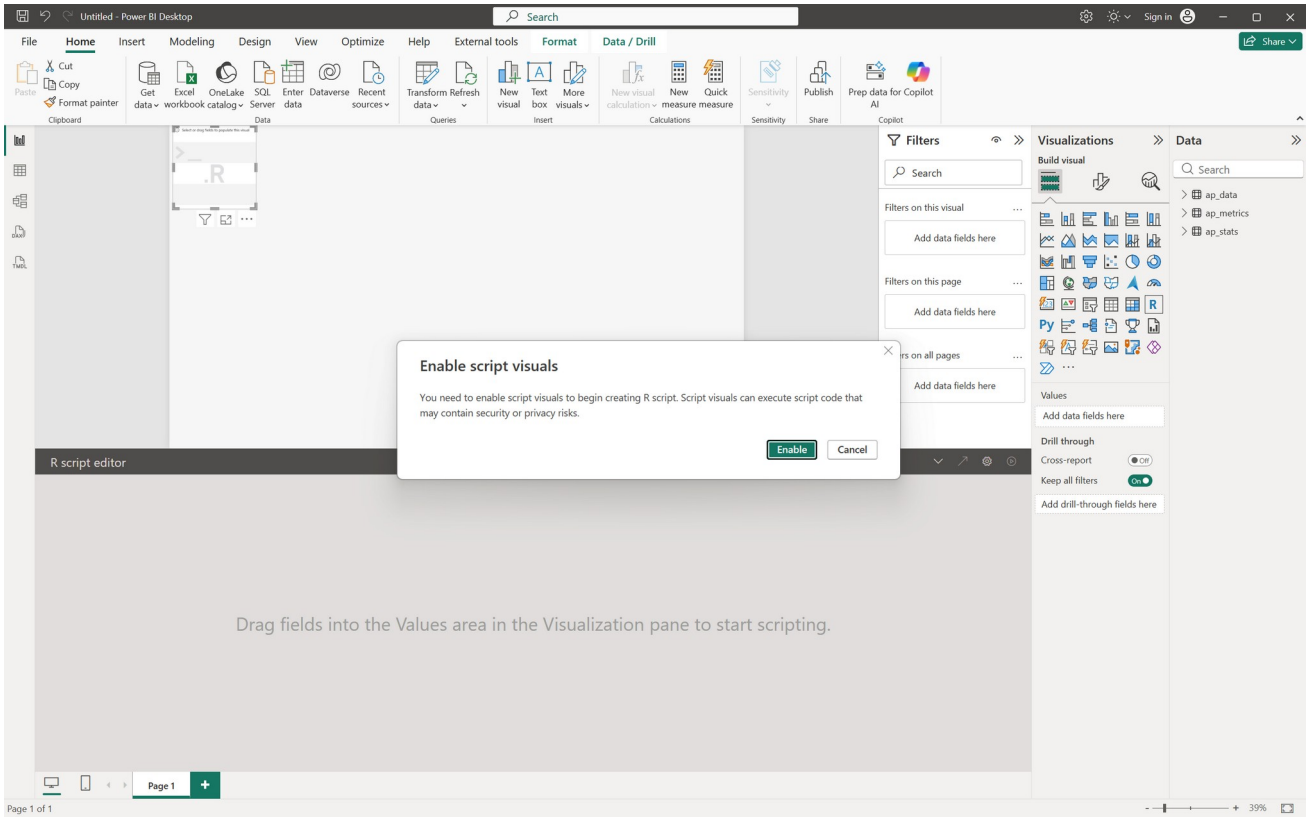


Figure 2: Power BI asking to enable script visuals. The three tables from the R data source are already in the Data pane

4. Task 2: Importing the dataset through an R script

The data was brought into Power BI using Home > Get data > More > Other > R script. Power BI runs the script and lets you load every data frame that is left in memory at the end. The script below builds three tables.

- **ap_data** has one row per month with Date, Passengers, Year, MonthNo, MonthName, Quarter, Season and Type. It also has the 12 forecast rows for 1961 with their 95% bounds (Lo95 and Hi95)
- **ap_stats** holds the summary statistics
- **ap_metrics** holds the ARIMA model details and its test accuracy

```
library(forecast)
ap <- AirPassengers
ap_data <- data.frame(
  Date       = seq(as.Date("1949-01-01"), by = "month", length.out = length(ap)),
  Passengers = as.numeric(ap))

# helper columns for slicers and drill-down
ap_data$Year      <- as.integer(format(ap_data$Date, "%Y"))
ap_data$MonthNo   <- as.integer(format(ap_data$Date, "%m"))
ap_data$MonthName <- month.abb[ap_data$MonthNo]
ap_data$Quarter   <- paste0("Q", (ap_data$MonthNo - 1) %/% 3 + 1)
ap_data$Season    <- ifelse(ap_data$MonthNo %in% 6:8, "Peak (Jun-Aug)", "Off-peak")
ap_data$Type      <- "Actual"
```

5. Task 3: Cleaning missing values and duplicates

Before any analysis the data was checked for gaps and repeated months. The script counts both and has a fix ready in case either shows up. Missing months would be filled using na.interp from the forecast package which respects the trend and the seasonal pattern. Duplicate dates would simply be dropped.

```
cat("Missing values:", sum(is.na(ap_data$Passengers)), "\n")    # 0
cat("Duplicate dates:", sum(duplicated(ap_data$Date)), "\n")    # 0
ap_data <- ap_data[!duplicated(ap_data$Date), ]
if (any(is.na(ap_data$Passengers))) {
  ap_data$Passengers <- as.numeric(na.interp(ts(ap_data$Passengers, frequency = 12)))
}
```

The check found 0 missing values and 0 duplicate dates so all 144 rows were kept as they are. In Power Query the column types were then set to Date for Date, Whole number for Year and MonthNo, Decimal number for Passengers and Text for the rest. MonthName was sorted by MonthNo so months show in calendar order rather than A to Z.

6. Task 4: Running R scripts inside Power BI

R is used in two places in this report. The first is the data source script from Task 2 which runs every time the data is refreshed. The second is the R script visual. For each R visual the needed fields are dragged into Values and Power BI passes them to R as a data frame called dataset. Passengers is set to Don't summarize and Date is always included so that each month stays as its own row. Power BI removes duplicate rows before running the script so without Date some months with the same value would be lost.

Each R visual also has a visual level filter of Type = Actual. This keeps the 1961 forecast rows out of the charts that are meant to show history only.

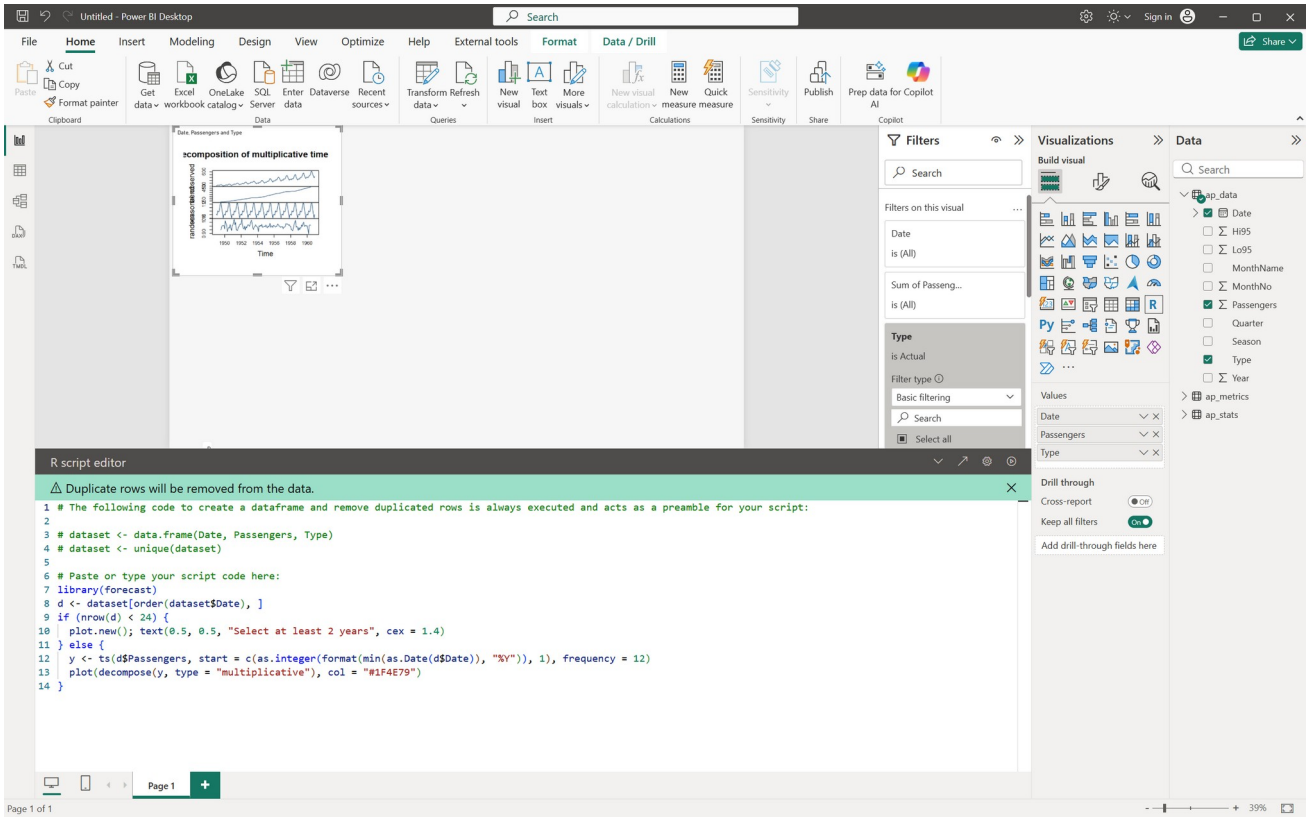


Figure 3: R visual 1 being built. Fields Date, Passengers and Type are in Values, the Type filter is set to Actual and the script is in the R script editor

7. Task 5: Summary statistics

The ap_stats table was built in R with summary(), mean() and sd(). All values are in thousands of passengers.

Statistic	Value
Count	144 months
Mean	280.30
Median	265.50
Standard deviation	119.97
Minimum	104 (November 1949)
First quartile	180.00
Third quartile	360.50
Maximum	622 (July 1960)

The mean is a little higher than the median which means the data leans to the right. This is because the later years have much bigger numbers than the early years. The standard deviation is large compared to the mean which tells us the series is far from flat. The busiest month carried about six times as many passengers as the quietest one.

8. Task 6: Visualizations in R

Four R visuals were made. Each one shows something that a normal Power BI chart cannot show easily.

R1: Seasonal decomposition

The decompose() function splits the series into trend, seasonal and random parts. A multiplicative model was used because the summer peaks get bigger as traffic grows. If fewer than 24 months are selected the visual shows a short message instead of an error.

```
library(forecast)
d <- dataset[order(dataset$Date), ]
if (nrow(d) < 24) {
  plot.new(); text(0.5, 0.5, "Select at least 2 years", cex = 1.4)
} else {
  y <- ts(d$Passengers, start = c(as.integer(format(min(as.Date(d$Date)), "%Y")), 1),
frequency = 12)
  plot(decompose(y, type = "multiplicative"), col = "#1F4E79")
}
```

R2: ARIMA forecast for 1961

This visual fits the seasonal ARIMA model on whatever years are selected and plots the next 12 months with the 80% and 95% bands.

```
library(forecast); library(ggplot2)
d <- dataset[order(dataset$Date), ]
y <- ts(d$Passengers, start = c(as.integer(format(min(as.Date(d$Date)), "%Y")), 1), frequency
= 12)
fit <- Arima(y, order = c(0,1,1), seasonal = c(0,1,1), lambda = 0)
autoplot(forecast(fit, h = 12)) +
  labs(title = "1961 forecast - ARIMA(0,1,1)(0,1,1)[12]", x = NULL, y = "Passengers (000s)") +
  theme_minimal(base_size = 13)
```

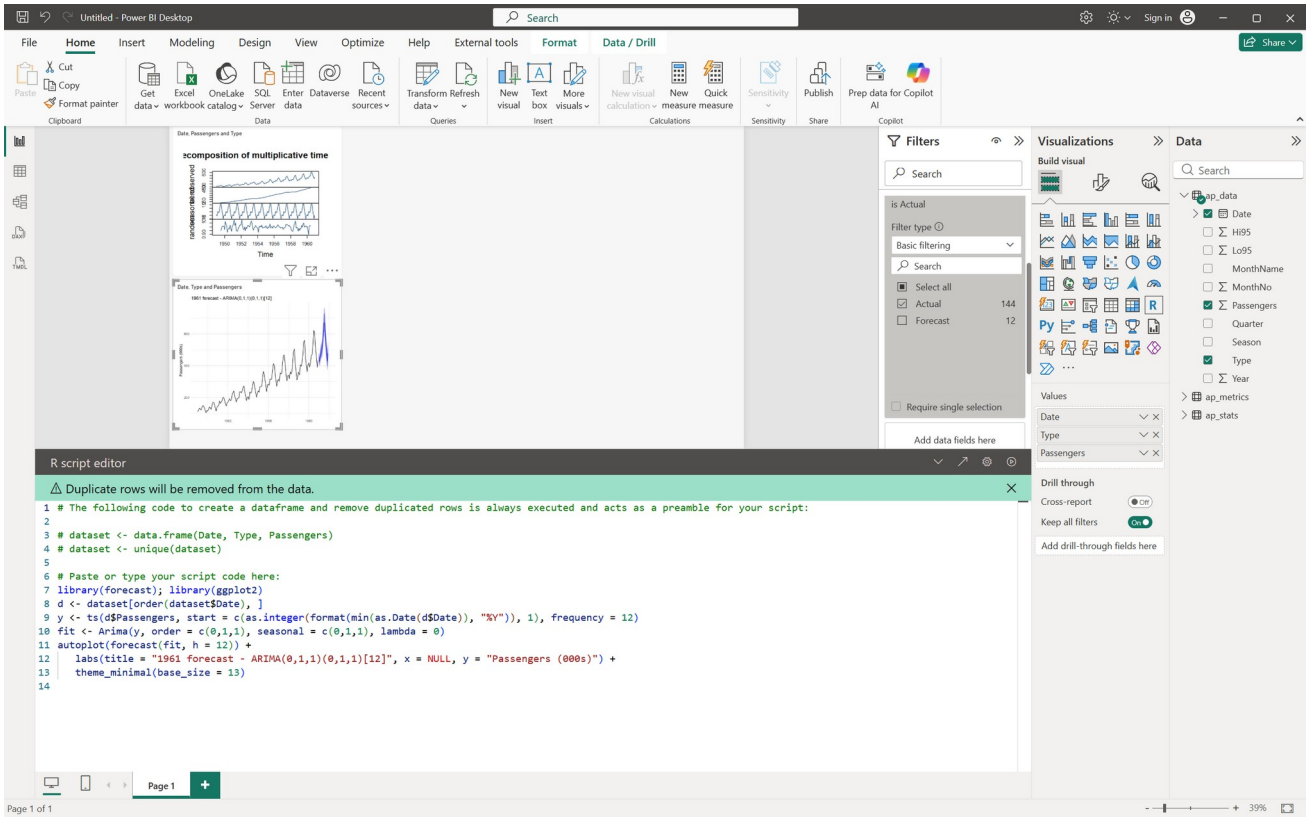


Figure 4: R visual 2 showing the ARIMA forecast for 1961 with its confidence bands

R3: Monthly boxplot

This boxplot shows the spread of every calendar month across all twelve years. June to August are filled in orange so the peak season stands out.

```
library(ggplot2)
dataset$MonthName <- factor(dataset$MonthName, levels = month.abb)
ggplot(dataset, aes(MonthName, Passengers, fill = MonthNo %in% 6:8)) +
  geom_boxplot(show.legend = FALSE) +
  scale_fill_manual(values = c("#9DB9D3", "#E07B39")) +
  labs(title = "Passengers by month (orange = Jun-Aug peak)", x = NULL, y = "Passengers
(000s)") +
  theme_minimal(base_size = 13)
```

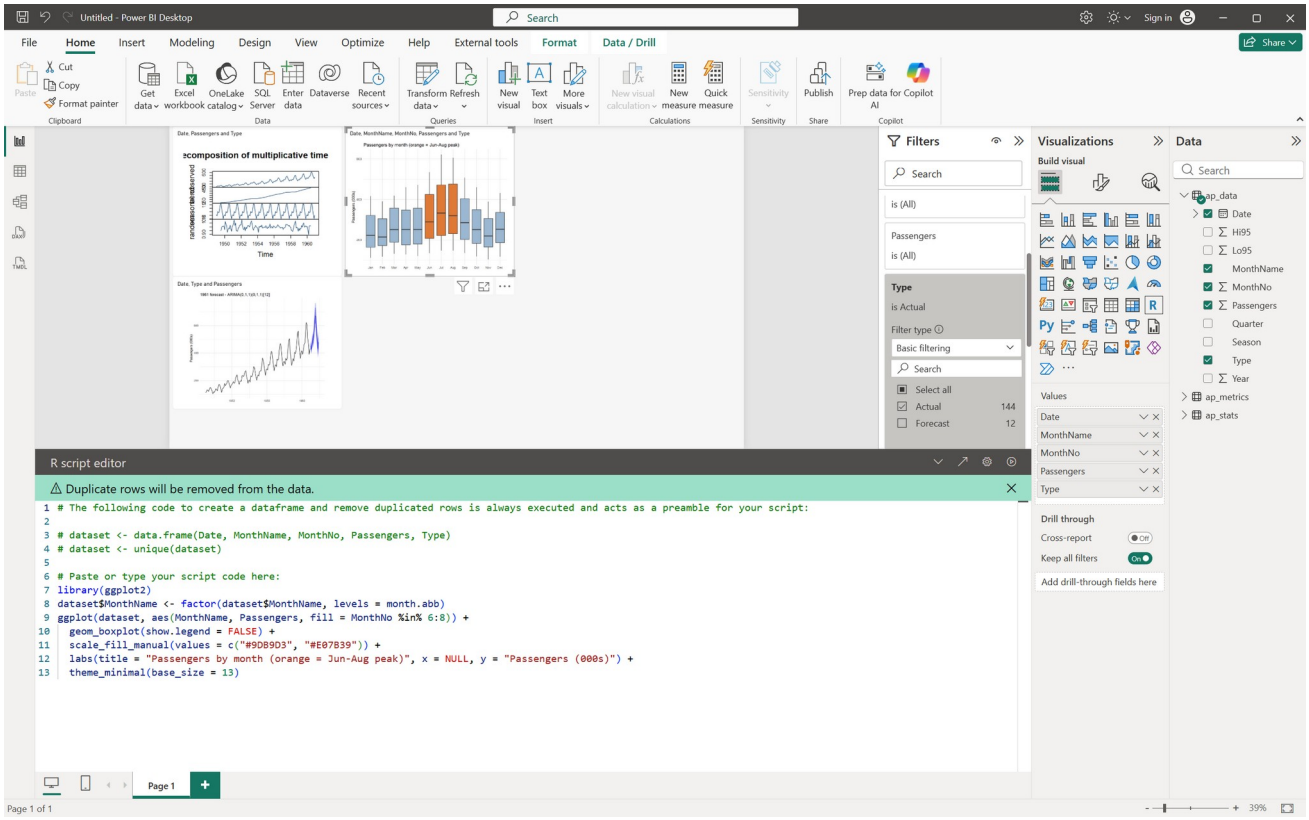


Figure 5: R visual 3, the monthly boxplot, with fields Date, MonthName, MonthNo, Passengers and Type

R4: Year by month heatmap

The heatmap puts years on one axis and months on the other. Darker cells mean more passengers so both the yearly growth and the summer peak can be seen in one picture.

```
library(ggplot2)
dataset$MonthName <- factor(dataset$MonthName, levels = month.abb)
ggplot(dataset, aes(MonthName, factor(Year), fill = Passengers)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = Passengers), size = 3) +
  scale_fill_gradient(low = "#EAF1F8", high = "#1F4E79") +
  labs(title = "Monthly passengers heatmap", x = NULL, y = NULL) +
  theme_minimal(base_size = 13)
```

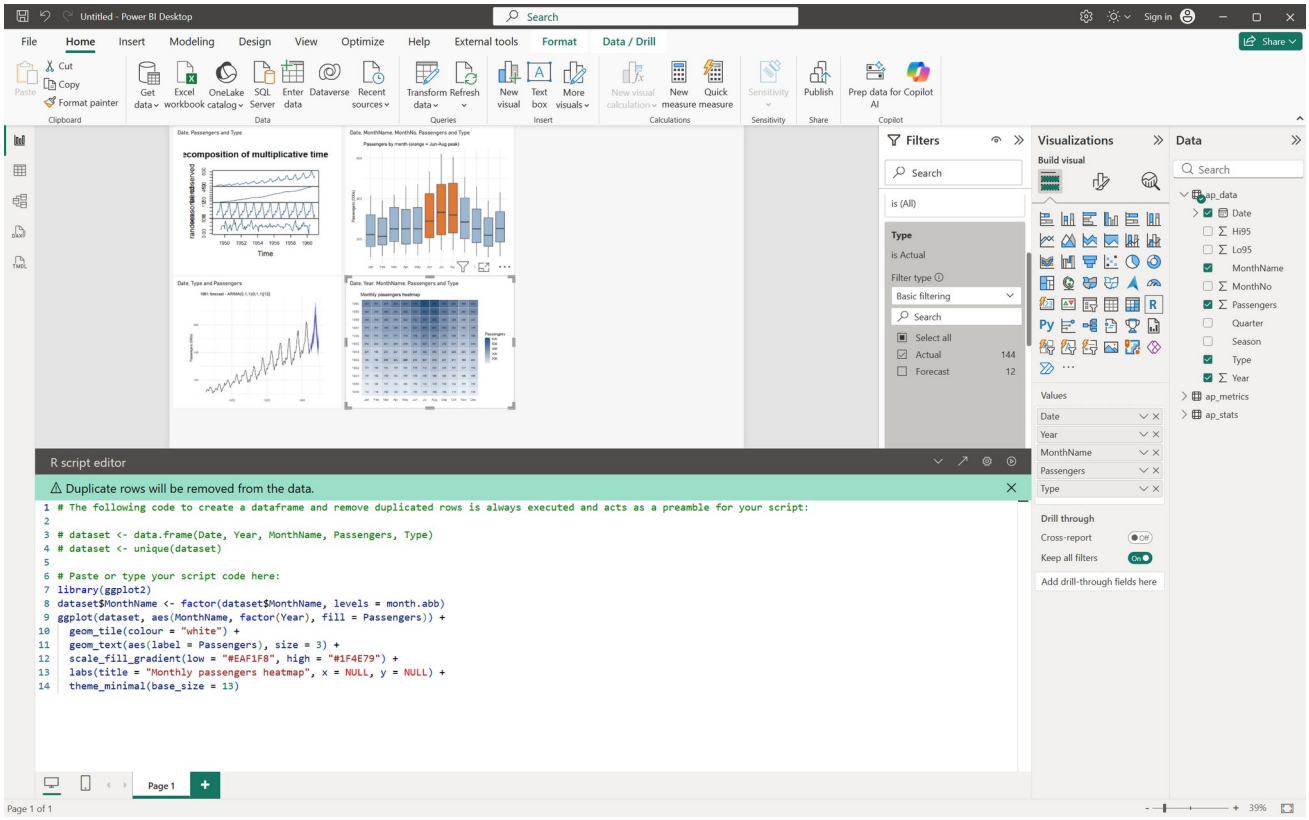


Figure 6: All four R visuals on the canvas with the heatmap script open in the R script editor

9. Task 7: Statistical model: seasonal ARIMA

The technique used is a seasonal ARIMA model, written as ARIMA(0,1,1)(0,1,1)[12]. It is also called the airline model because it was first made famous on this exact dataset. The model is fitted on the log of passengers (lambda = 0). Taking the log turns the growing summer peaks into peaks of the same size which the model can handle well.

In simple words the model takes the change from the same month last year and from the month before. It then corrects its next guess using its last two errors, one from last month and one from twelve months back.

To check the model fairly it was first trained on 1949 to 1959 only. It was then asked to predict 1960 and the guesses were compared with the real 1960 numbers. After that the model was fitted on all 12 years to forecast 1961.

```
train  <- window(ap_ts, end = c(1959, 12))
test   <- window(ap_ts, start = c(1960, 1))
fit_tr <- Arima(train, order = c(0,1,1), seasonal = c(0,1,1), lambda = 0)
acc    <- accuracy(forecast(fit_tr, h = 12), test)

fit <- Arima(ap_ts, order = c(0,1,1), seasonal = c(0,1,1), lambda = 0)
fc  <- forecast(fit, h = 12, level = c(80, 95))
lb  <- Box.test(residuals(fit), lag = 24, type = "Ljung-Box", fitdf = 2)
```

Model output	Value
MA(1) coefficient	-0.4018
Seasonal MA(1) coefficient	-0.5569
AIC	-483.4
Test MAPE (1960)	2.90%
Test RMSE (1960)	18.59 thousand
Ljung-Box p-value	0.233
Forecast total for 1961	6,240 thousand (+9.2% on 1960)
Forecast for July 1961	670 thousand (95% range 589 to 762)

On the year it had never seen the model was off by only 2.9% on average. The Ljung-Box p-value of 0.233 is above 0.05 which means the leftover errors look like random noise. So the model has picked up the trend and the seasonal pattern and there is no clear pattern left that it missed.

DAX measures for the KPI cards

A separate _Measures table was made in Power BI to hold the measures used by the cards and charts. A few of them are shown below.

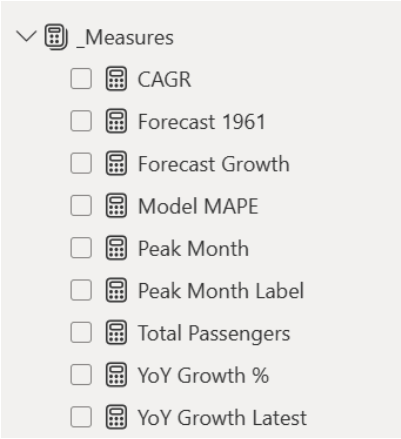


Figure 7: The DAX measures created in the _Measures table

```
Total Passengers = CALCULATE(SUM(ap_data[Passengers]), ap_data[Type] = "Actual")

Peak Month = CALCULATE(MAX(ap_data[Passengers]), ap_data[Type] = "Actual")

CAGR =
VAR firstY = CALCULATE(MIN(ap_data[Year]), ap_data[Type] = "Actual")
VAR lastY  = CALCULATE(MAX(ap_data[Year]), ap_data[Type] = "Actual")
VAR v0 = CALCULATE(SUM(ap_data[Passengers]), ap_data[Year] = firstY, ap_data[Type] = "Actual")
VAR v1 = CALCULATE(SUM(ap_data[Passengers]), ap_data[Year] = lastY, ap_data[Type] = "Actual")
RETURN DIVIDE(v1, v0) ^ DIVIDE(1, lastY - firstY) - 1

YoY Growth % =
VAR yr   = MAX(ap_data[Year])
VAR cur  = CALCULATE(SUM(ap_data[Passengers]), ap_data[Type] = "Actual")
VAR prev = CALCULATE(SUM(ap_data[Passengers]), ap_data[Type] = "Actual",
                     FILTER(ALL(ap_data[Year]), ap_data[Year] = yr - 1))
RETURN IF(NOT ISBLANK(prev), DIVIDE(cur - prev, prev))
```

10. Task 8: The interactive dashboard

Everything was put together on one 2560 by 1440 page with a navy and orange theme. The page has a title strip with slicers at the top, a row of KPI cards, two rows of four charts and an insights strip at the bottom.

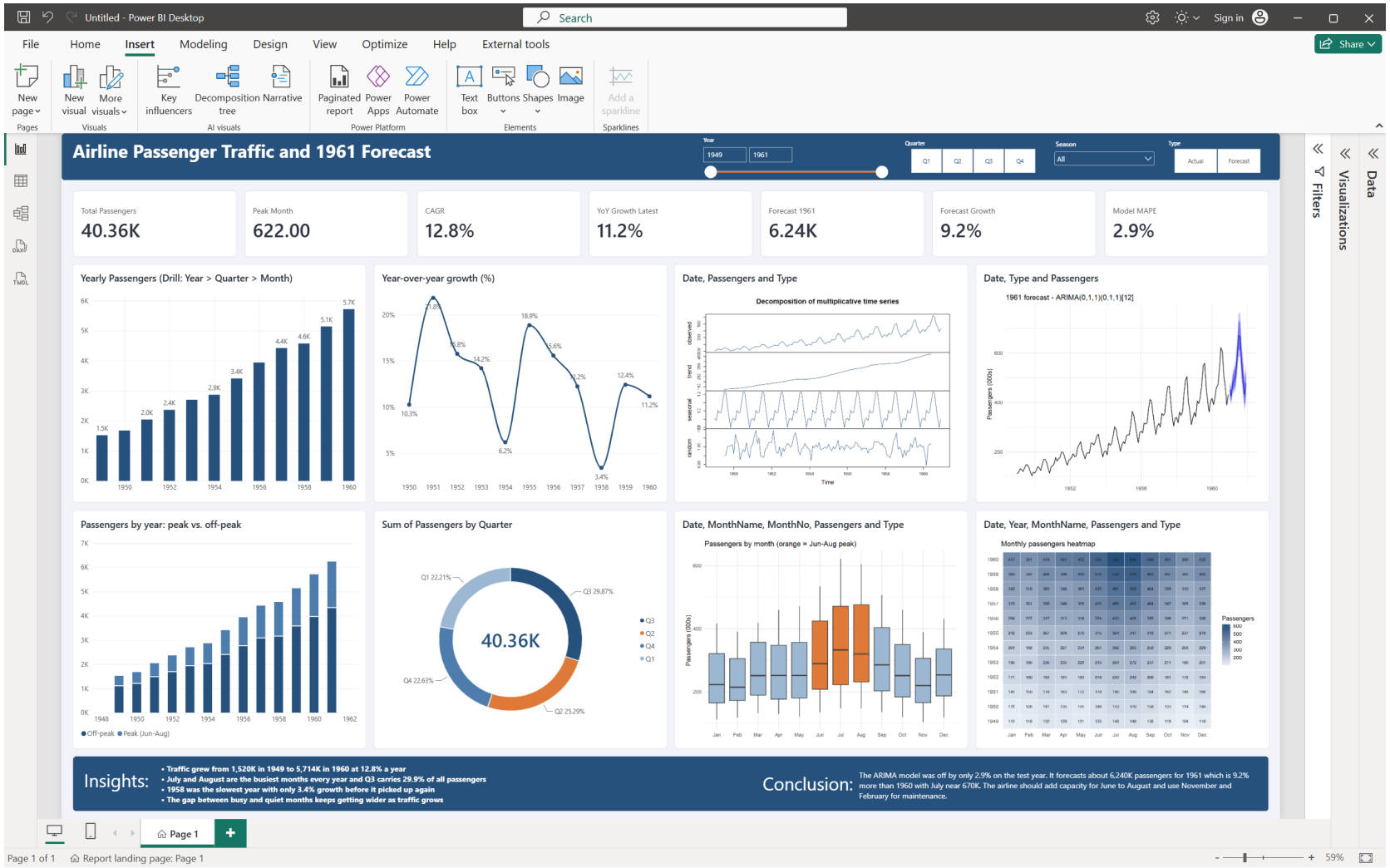


Figure 8: Final dashboard showing the seven KPI cards, four Power BI visuals, four R visuals, slicers and the insights strip

KPI cards (7)

Card	Value	What it tells us
Total Passengers	40.36K	All passengers from 1949 to 1960 in thousands
Peak Month	622	Busiest single month (July 1960)
CAGR	12.8%	Average yearly growth from 1949 to 1960
YoY Growth Latest	11.2%	Growth of 1960 over 1959
Forecast 1961	6.24K	Total passengers the model expects in 1961
Forecast Growth	9.2%	Expected growth of 1961 over 1960
Model MAPE	2.9%	Average error of the model on the 1960 test year

Power BI visuals (4)

- **Yearly Passengers column chart:** uses a Year > Quarter > Month hierarchy so the user can drill down from a year to its quarters and then its months
- **Year-over-year growth line chart:** shows the growth rate of each year using the YoY Growth % measure
- **Peak vs off-peak stacked column chart:** splits every year into June to August and the rest of the year
- **Passengers by quarter donut chart:** shows the share of each quarter with total passengers in the centre

R visuals (4)

Seasonal decomposition, ARIMA forecast, monthly boxplot and year by month heatmap as described in Task 6.

Interactivity

- **Slicers:** Year (between slider), Quarter (tiles), Season (dropdown) and Type (tiles). All of them sit in the title strip
- **Filters:** visual level filters of Type = Actual keep forecast rows out of the history charts and a page filter removes blank dates
- **Drill-down:** the yearly column chart drills from Year to Quarter to Month using the drill arrows on the visual
- **Cross filtering:** clicking a bar or a donut slice filters the other Power BI visuals. R visuals redraw when a slicer changes. The Quarter and Season slicers were turned off for the decomposition and ARIMA visuals using Edit interactions because those two need full consecutive years

11. Task 9: Business insights

1. **Strong and steady growth.** Yearly traffic went from 1,520 thousand in 1949 to 5,714 thousand in 1960. That is almost four times bigger and works out to 12.8% growth every year.
2. **Summer is the busiest time.** July and August are the top months in every single year. July is about 23% above an average month while November is about 20% below it. Q3 alone carries 29.9% of all passengers.
3. **Growth slowed for a while.** 1958 was the weakest year with only 3.4% growth. Traffic picked up again in 1959 and 1960 which shows the dip was short.
4. **The peak keeps getting bigger.** The gap between busy and quiet months grows as traffic grows. This is why a multiplicative model and a log scale were the right choice.

5. **1961 will be another record year.** The model expects about 6,240 thousand passengers in 1961 which is 9.2% more than 1960. July 1961 alone may reach around 670 thousand.

What the airline should do

- Add extra flights and staff for June to August 1961 since those three months may need about 9% more seats than in 1960
- Plan aircraft maintenance and staff leave in November and February when demand is lowest
- Use the 95% upper bound (762 thousand for July) as the safe limit when booking extra capacity
- Refit the model every month as new data arrives so the forecast stays accurate

12. Task 10: Conclusion

This practical showed how R and Power BI work well together. R handled the parts that need real statistics like cleaning the data, splitting the series into trend and season and building a forecast model that was tested on a year it had not seen. Power BI handled the parts that need interaction like KPI cards, slicers, drill-down and cross filtering on one clean page.

The seasonal ARIMA model was accurate with only 2.9% error on the 1960 test year and its errors showed no leftover pattern. It forecasts about 6,240 thousand passengers for 1961 with the summer months once again the busiest. One limit of R visuals in Power BI is that they are static images. They react to slicers but cannot be clicked to drill down or filter other visuals. Using Power BI visuals for the interactive parts and R visuals for the analysis parts solved this.